National University of Lesotho
Department of Mathematics and Computer Science

CS4433 Software Engineering
Individual Final Project

Moderated Learning Management System

Project Management Plan

Student Name: MAKHOTSO RAMOCHELA

Student Number: 202321185

1. Introduction

1.1 Purpose of this document

This is my project management plan for the Moderated Learning Management System (LMS). It covers the Phase 1 tasks in the assignment brief: a feasibility study, a work breakdown structure, a schedule with dependencies, milestones and the critical path, and a risk register. I will use it as the baseline to measure my progress against, and I will update the risk register at each phase gate.

1.2 The project in brief

A secondary school needs a system to manage courses, assignments and marking. The users are students, lecturers, Heads of Department (HoDs) and an administrator and each of them may only do what their role allows. The key rule is that students must not see a mark until the HoD has moderated it and released it. The system will be a C++17 application that keeps its data in SQLite and works offline on a school computer. I am doing the project on my own. It follows the Waterfall model with V-Model testing, with a review with my supervisor at each of three gates: Gate 1 (requirements), Gate 2 (design) and Gate 3 (testing and release of version 1.0).

1.3 Planning assumptions

- My working time runs from Monday 28 September 2026 to Friday 6 November 2026. That is six weeks, or 30 working days (Monday to Friday). The date on the brief is 7 November 2026, which is a Saturday, so I plan to finish on the Friday and keep the Saturday as an emergency reserve.
- The brief describes a seven-week project, but I have six weeks. The Maintenance and Completion phases are therefore done in Week 6, straight after release v1.0. The order of the phases and the three gates stays the same.
- I can work about 6 hours on each working day, which is 30 hours a week and 180 hours in total. Weekends are not scheduled and are kept as spare time.
- I work alone, so tasks are done one after another with no overlaps. Effort is estimated in hours.
- No system code is written before Gate 2. In Week 1 I only set up the repository and build files, which is project set-up and not implementation.

2. Feasibility study

2.1 Technical feasibility

All the tools I need are free and widely used: a C++17 compiler, CMake, SQLite, Git with a hosted repository, GoogleTest (or Catch2) for unit tests, and cppcheck or clang-tidy for static

analysis. I chose SQLite because it is a library that stores the whole database in one file, so no database server has to be installed and run. This suits the IT officer's point that the school's computers are old and the internet is unreliable. A database holding 800 students with their courses and submissions is small for SQLite. I also considered PostgreSQL, but it needs a server that a school technician would have to install and maintain, and the system does not need its extra features. The SQLite documentation recommends a client/server database when many programs use the same data over a network (SQLite, n.d.), so I am assuming one installation on one school computer for version 1.0. I will confirm this in the design phase.

Passwords will be stored as salted hashes and never as plain text. I will use the libsodium library and its Argon2id password hashing instead of writing my own code (libsodium, n.d.). Password hashing is slow on purpose, so I will choose settings that keep login fast on old computers and time a login early in implementation. I plan a console interface first, because a graphical toolkit such as Qt would take longer to learn and could be hard to install on the school's computers. I will confirm the interface choice at Gate 2.

The biggest technical concern is not the tools but that I am the only developer on a fairly large C++ program. To manage this, I will build the system in small steps (data layer, then login, then submissions, then business rules), as shown in the schedule. In Week 1 I will also check that an empty CMake project can link SQLite and libsodium and run one test, so that build problems show up early.

2.2 Operational feasibility

The Principal said the system must be simple for teachers who are not IT experts. A menu that only shows the actions allowed for the user's role keeps the screens short, and I will turn "simple" into measurable targets in the SRS. The moderation step changes how marking works: lecturers can mark work but cannot release marks, and the HoD releases them. This is what the Principal and the HoD asked for. To meet the lecturers' wish to see results quickly, a lecturer will see the status of each submission (for example "marked" or "released") and will be notified when the HoD releases the marks. The system works offline, as the IT officer asked. A short user manual for each role should be enough training.

2.3 Economic feasibility

There is no software cost. SQLite is in the public domain, and GoogleTest and libsodium are open-source libraries that can be used freely. The school does not need new computers because SQLite and a console program run well on old hardware. The main cost is my own time: the plan uses 168 of the 180 hours I have available. The case study gives no figures for what the school spends now, so I have not estimated savings in money. The benefits are practical: marks and feedback are kept in one place, moderation is enforced by the system, and every mark change is recorded.

2.4 Legal feasibility

The system will hold learners' names, course enrolments, submissions and marks, and many learners are under 18. The Lesotho Data Protection Act, 2011 therefore applies (Lesotho, 2011), together with the school's own rules. In practice this means collecting only the data that is needed, letting each role see only what it should, keeping the data secure, and being able to show who changed a mark. The design will do this with role-based access, salted password hashes, a small user profile, an audit log of mark changes, and the rule that students cannot see marks before release. I will check the license of each library and list them in the repository. All the work will be my own, and I will reference any sources I use. A fuller discussion is in the professional, legal and ethical issues section of the final report.

2.5 Schedule feasibility

I have six weeks, which is 30 working days from Monday 28 September to Friday 6 November, while the brief describes seven weeks. This means the Maintenance and Completion phases are squeezed into Week 6. The plan uses 168 of the 180 hours I can give, so there are only 12 spare hours. The gate dates and the final deadline cannot move, so what can change is the amount of work, not the deadline. At Gate 1 I will give every requirement a MoSCoW priority, and if I fall behind I will drop Could and then Should requirements first. Late changes are expensive in the Waterfall model (Sommerville, 2016), so the gate reviews and the traceability matrix are important for catching problems early.

2.6 Summary and recommendation

Table 1: Summary of the feasibility study

Area	Main points	Result
Technical	Free, well-known tools. SQLite works offline on old computers. One installation assumed for v1.0. Skill risk reduced by building in small steps.	Feasible
Operational	Role-based menus keep the system simple. Moderation follows the Principal's and HoD's requirements.	Feasible
Economic	No licence or hardware cost. About 168 hours of my time.	Feasible
Legal	Learner data is covered by the Data Protection Act, 2011. The design includes access control, hashing and an audit log.	Feasible
Schedule	30 working days and only 12 spare hours. Scope is reduced first if needed.	Feasible, but tight

Recommendation: All five areas are feasible and the main concern is time, not technology. Therefore, the project will be done.

3. Work breakdown structure

I followed the phases in the assignment brief and split the project into 7 phases and 28 tasks. Each task produces one deliverable, so it is easy to tell when it is finished. Table 2 lists the tasks and their deliverables. The task numbers are used in the schedule in Section 4.

Table 2: Work breakdown structure

No.	Task	Deliverable
1	**Planning and management**	
1.1	Feasibility study	Feasibility report with a recommendation
1.2	Set up Git repository, CMake build and CI skeleton	Repository with branching plan, build files and a working CI run
1.3	Work breakdown structure and Gantt chart	Work breakdown structure, Gantt chart and critical path
1.4	Risk register	Risk register (10 risks)
1.5	Check and finish the project management plan	Finished project management plan (Phase 1 deliverable)
2	**Requirements engineering**	
2.1	Stakeholder analysis and conflict log	Stakeholder list, conflicts found and how each was resolved
2.2	Software Requirements Specification (SRS)	SRS with at least 12 functional and 6 measurable non-functional requirements, with MoSCoW priorities
2.3	Use case diagram and three use case specifications	Use case diagram (4 actors) and 3 detailed specifications including Submit assignment
2.4	Requirements review and first traceability matrix	Reviewed requirements and first traceability matrix (Gate 1)
3	**System design**	
3.1	Architecture and component diagram	Architecture style, component diagram and a rejected alternative
3.2	Class diagram, design patterns and SOLID notes	Class diagram with at least two design patterns and SOLID notes
3.3	State machine, activity and sequence diagrams	State machine, swimlane activity and sequence diagrams
3.4	Database design (ERD, data dictionary, SQL schema)	ERD in 3NF, data dictionary and SQL schema (Gate 2)
4	**Implementation**	
4.1	Data-access layer and SQLite schema	Data-access layer with parameterised queries; schema created in SQLite
4.2	Login, password hashing, roles and menus	Login with salted hashes, abstract User class with four roles, role menus
4.3	Courses, assignments and submissions	Courses, assignments, submissions, resubmission and feedback
4.4	Late penalty, extensions, moderation and audit log	Penalty rule (Strategy), extensions, moderation and release, audit log
5	**Verification and validation**	
5.1	Test plan	Test plan with V-Model levels, entry and exit criteria
5.2	Unit and integration tests, coverage report	Automated tests run in CI and a coverage report (target 70%)
5.3	System and acceptance tests	System tests from the use cases, including exception flows, and a defect log
5.4	Fix defects and retest	Defects fixed and retested, results recorded
6	**Quality assurance and deployment**	
6.1	Static analysis, code metrics and self code review	Static analysis report, code metrics and code review record

No.	Task	Deliverable
6.2	Deployment diagram, installation guide, manuals, release notes	Deployment diagram, installation guide, user manuals, release notes; v1.0 tagged after task 5.4
7	**Maintenance and completion**	
7.1	CR1 classification and impact analysis	Classification and impact analysis of CR1
7.2	Implement CR1 (release v1.1)	Release v1.1 and updated design documents
7.3	New and regression tests, tag v1.1	New tests, regression test results and v1.1 tag
7.4	Professional, legal and ethical issues section	Professional, legal and ethical issues section
7.5	Final report check and viva preparation	Final report (PDF) checked and viva preparation done

4. Project schedule

4.1 How the schedule was built

Because I work alone, I can only do one task at a time. I therefore placed the tasks one after another in the order shown in Table 3, with no overlaps. Time is counted in hours, with 6 working hours in a day. A task that is not a multiple of 6 hours ends part-way through a day, and the next task starts later on that same day. The work is split into four stages. Each stage ends at a gate (or at the final date), and any hours left over before the gate are kept as spare time. Here is how to read Table 3:

- **Start, End and Hours:** the first and last day of the task and my estimate of the effort in hours.
- **Depends on:** the tasks that must be finished before this one can start. A dash means it can start straight away. Every task comes after the tasks it depends on, so these tasks always appear above it in the table.
- **Float:** the number of spare hours before the next gate. If the tasks in a stage together take more than their estimates by more than the float, the gate date is missed. The float is shared by all the tasks in a stage and is not given to each task separately. For example, Stage 1 has 6 hours of spare time (the whole of Friday 9 October, which is the Gate 1 day), so tasks 1.1 to 2.4 each show a float of 6 hours. A task with a float of 0 is on the critical path. These tasks are marked with *.

Table 3: Schedule, dependencies and float (* = critical task)

ID	Task	Start	End	Hours	Depends on	Float (hours)
Stage 1: Planning and requirements (ends at Gate 1, Fri 9 Oct). Spare time: 6 hours						
1.1	Feasibility study	28 Sep	29 Sep	7	-	6
1.2	Set up Git repository, CMake build and CI skeleton	29 Sep	29 Sep	4	-	6
1.3	Work breakdown structure and Gantt chart	29 Sep	30 Sep	5	1.1	6
1.4	Risk register	30 Sep	1 Oct	4	1.1	6

ID	Task	Start	End	Hours	Depends on	Float (hours)
1.5	Check and finish the project management plan	1 Oct	1 Oct	2	1.3, 1.4	6
2.1	Stakeholder analysis and conflict log	1 Oct	2 Oct	8	1.1	6
2.2	Software Requirements Specification (SRS)	5 Oct	6 Oct	11	2.1	6
2.3	Use case diagram and three use case specifications	6 Oct	8 Oct	9	2.1	6
2.4	Requirements review and first traceability matrix	8 Oct	8 Oct	4	2.2, 2.3, 1.5	6
Stage 2: Design (ends at Gate 2, Fri 16 Oct). Spare time: 2 hours						
3.1	Architecture and component diagram	12 Oct	12 Oct	4	2.4	2
3.2	Class diagram, design patterns and SOLID notes	12 Oct	13 Oct	8	3.1	2
3.3	State machine, activity and sequence diagrams	14 Oct	15 Oct	7	2.4	2
3.4	Database design (ERD, data dictionary, SQL schema)	15 Oct	15 Oct	5	3.1	2
5.1	Test plan	16 Oct	16 Oct	4	3.2, 3.3	2
Stage 3: Implementation, testing and release (ends at Gate 3 and v1.0, Tue 3 Nov). Spare time: 4 hours						
4.1	Data-access layer and SQLite schema	19 Oct	20 Oct	7	3.2, 3.3, 3.4, 1.2	4
4.2	Login, password hashing, roles and menus	20 Oct	21 Oct	8	4.1	4
4.3	Courses, assignments and submissions	21 Oct	23 Oct	10	4.2	4
4.4	Late penalty, extensions, moderation and audit log	23 Oct	26 Oct	11	4.3	4
5.2	Unit and integration tests, coverage report	27 Oct	28 Oct	7	4.4, 5.1	4
5.3	System and acceptance tests	28 Oct	29 Oct	8	4.4, 5.2	4
5.4	Fix defects and retest	29 Oct	30 Oct	5	5.3	4
6.1	Static analysis, code metrics and self code review	30 Oct	30 Oct	4	4.4	4
6.2	Deployment diagram, installation guide, manuals, release notes	2 Nov	3 Nov	8	4.4	4
Stage 4: Change request, final report and viva preparation (ends Fri 6 Nov). Spare time: 0 hours						
7.1 *	CR1 classification and impact analysis	4 Nov	4 Nov	3	5.4, 6.1, 6.2	0
7.2 *	Implement CR1 (release v1.1)	4 Nov	5 Nov	4	7.1	0
7.3 *	New and regression tests, tag v1.1	5 Nov	5 Nov	4	7.2	0
7.4 *	Professional, legal and ethical issues section	5 Nov	6 Nov	3	6.2	0
7.5 *	Final report check and viva preparation	6 Nov	6 Nov	4	7.3, 7.4	0

4.2 Gantt chart

Figure 1 shows the same schedule as a Gantt chart. Black bars are critical tasks, grey bars are tasks with float and the hatched bars show the spare time before each gate.

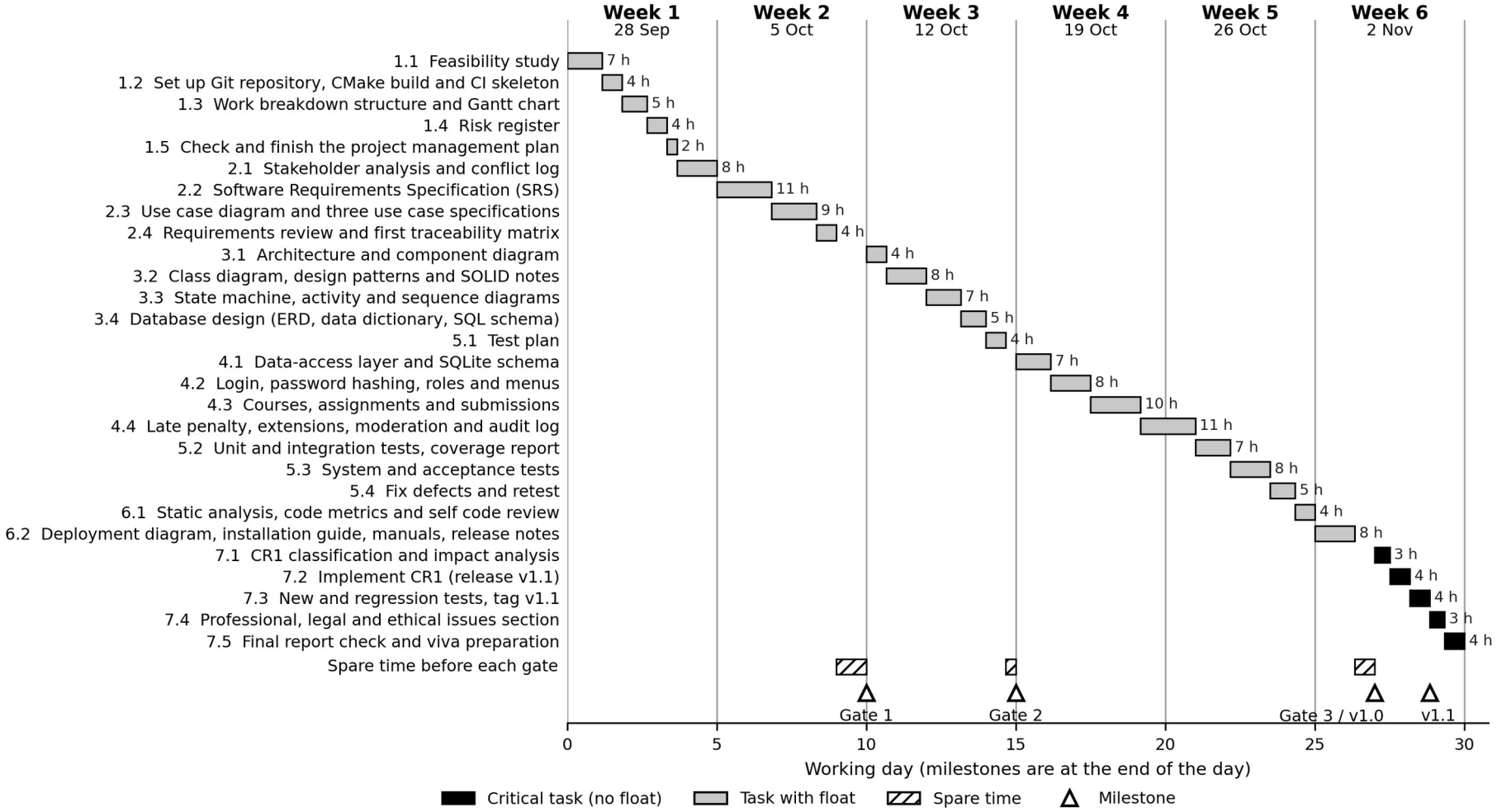


Figure 1: Gantt chart of the project schedule

4.3 Milestones

Table 4: Milestones

Milestone	Date	What must be ready
Gate 1	Friday 9 October	Stakeholder analysis, SRS and use case model reviewed with my supervisor; first traceability matrix; project plan finished.
Gate 2	Friday 16 October	Architecture, class, behavioral and database designs finished and consistent with the SRS. Implementation may start only after this gate.
Gate 3 and release v1.0	Tuesday 3 November	All Must requirements built and tested, coverage reported, serious defects fixed, installation guide and manuals done, v1.0 tagged.
Release v1.1	Thursday 5 November	CR1 implemented, design documents updated, new and regression tests passed, v1.1 tagged.
Final check	Friday 6 November	Final report and repository checked and the viva demonstration prepared. Saturday 7 November is kept as a reserve.

4.4 Critical path

The critical path is the set of tasks with no float. Because I work alone, the schedule is really one line of tasks, and the only protection against delays is the spare time before each gate. The tasks with a float of 0 are:

7.1 → 7.2 → 7.3 → 7.4 → 7.5

These are done in that order in the last three working days (4 to 6 November). If any of them is late, the final check passes Friday 6 November. Saturday 7 November is my only reserve after that, and I do not plan to use it. Some other points about the schedule:

- Total spare time is 12 hours out of 180 (Stage 1: 6 hours, Stage 2: 2 hours, Stage 3: 4 hours). Stage 2 has 2 spare hours in 28 hours of work and Stage 3 has 4 spare hours in 72 hours of work, so these are the stages I will watch most closely. This matches the highest risk in Section 5 (R1).
- If a stage runs over, I will first use its spare hours, then drop Could and Should items, and only then use a weekend.
- Some tasks cannot be moved. Task 4.1 cannot start before Gate 2, and the change request tasks (7.1 to 7.3) cannot start before v1.0 is released, because CR1 comes from the school after release.
- Tasks 1.2, 5.1 and 7.4 depend on very few other tasks, so I can swap their order with other tasks in the same stage if that helps.

4.5 Weekly workload

Table 5: Planned hours compared with available hours

Week	Dates	Planned hours	Available hours	Spare hours
1	28 September to 2 October	30	30	0
2	5 to 9 October	24	30	6

Week	Dates	Planned hours	Available hours	Spare hours
3	12 to 16 October	28	30	2
4	19 to 23 October	30	30	0
5	26 to 30 October	30	30	0
6	2 to 6 November	26	30	4
Total		**168**	**180**	**12**

Weeks 1, 4 and 5 are fully used. The spare time is in Week 2 (Friday 9 October, Gate 1 day), Week 3 (2 hours) and Week 6 (4 hours before Gate 3). The weekends and Saturday 7 November are not planned.

5. Risk management

5.1 How the risks are rated

I rated each risk by how likely it is (L) and how much it would hurt the project (I), using a scale of 1 to 3 for both (Table 6). The score is L multiplied by I (Boehm, 1991). A score of 6 to 9 is high, 3 or 4 is medium and 1 or 2 is low.

Table 6: Rating scales

Value	Likelihood (L)	Impact (I)
1	Unlikely	Small: less than 1 day lost
2	Possible	Medium: 1 to 3 days lost or some rework
3	Likely	Serious: a gate or the viva is at risk, or work is lost

5.2 Risk register

Table 7 lists ten risks: four technical, three project and three people risks. I own all of them because I work alone, and my supervisor is involved where a decision needs their agreement.

Table 7: Risk register (baseline, end of Week 1)

No.	Risk (type)	L	I	Score	Action to reduce the risk and back-up plan	Owner
R1	Implementation takes longer than planned because this is my first large C++ project (Technical)	3	3	9	Build in small steps from Gate 2 (data layer, login, submissions, rules). Check hours used every Friday. If behind, drop Could and then Should items and use a weekend.	Student
R2	Requirements are unclear or conflict, which causes rework (Project)	3	2	6	Record every assumption in the conflict log. Confirm the late-penalty and mark-release rules with my supervisor at Gate 1. Accept later changes only through the change log.	Student, supervisor
R3	I am ill or have a personal problem (People)	2	3	6	Avoid leaving work to the last day. Tell my supervisor early. Follow the institution's procedure for extensions.	Student
R4	Power cuts or no internet (People)	3	2	6	Work offline and commit locally, then push when connected. Keep a copy of the work on a flash drive. Run the tests locally if CI is not reachable.	Student

No.	Risk (type)	L	I	Score	Action to reduce the risk and back-up plan	Owner
R5	Work is lost through disk failure or deleted files (Technical)	1	3	3	Push to the hosted repository every day. Copy the report files to a second place every Friday.	Student
R6	SQLite or libsodium will not build, or password hashing is too slow on old computers (Technical)	2	2	4	Link both libraries in an empty CMake project in Week 1. Choose lighter hashing settings and time a login. If libsodium fails, use another recognised library such as bcrypt.	Student
R7	Too little testing, so bugs are found late (Technical)	2	3	6	Write tests while coding. Run CI on every push. Check coverage against the 70% target each Friday. Keep tasks 5.3 and 5.4 for system testing and fixes.	Student
R8	Code, database and diagrams do not match each other (Project)	2	3	6	Keep the traceability matrix from Gate 1. Update the diagrams whenever the design changes. Do a consistency check before each gate.	Student
R9	Deadlines in my other courses take up project time (People)	3	2	6	Keep a fixed weekly timetable of 30 hours. Do the planning and design early. Talk to my supervisor if there is a clash.	Student
R10	Extra features are added and the scope grows (Project)	2	2	4	Use MoSCoW priorities and an out-of-scope list. Write new ideas in a future-work list instead of building them.	Student

R1 has the highest score (9). The implementation stage has only 4 spare hours, so I will check it every week. Seven risks score 6 or more. Several of them (R2, R8 and R10) are best dealt with in Weeks 1 and 2, which is why Gate 1 fixes the scope and records every assumption. Power cuts and internet problems (R4) are a real possibility, so I will work offline and commit often.

5.3 Updating the register

I will review and rescore the register at each gate, add new risks, close risks that are no longer a problem, and record the changes in Table 8.

Table 8: Risk register review log

Review	Date	Changes
Baseline	Thu 1 Oct	Ten risks identified and rated (Table 7).
Gate 1	Fri 9 Oct	To be updated at the gate review.
Gate 2	Fri 16 Oct	To be updated at the gate review.
Gate 3	Tue 3 Nov	To be updated at the gate review.

6. Tracking progress

Every Friday I will compare the hours I actually spent with Table 5 and check whether any task on the critical path is behind. A task counts as finished only when its deliverable exists, has been committed to the repository and has been checked against a short checklist. If I want to change the scope, for example by adding a requirement or dropping a Could item, I will write it in a change log with its effect on the schedule and the risks before accepting it. The plan itself is

kept in the repository, so every version made at a gate can be traced. The gate reviews with my supervisor are the outside check that the project is on course.

References

Boehm, B.W. (1991) 'Software risk management: principles and practices', IEEE Software, 8(1), pp. 32–41.

Lesotho (2011) Data Protection Act, 2011. Maseru: Government of Lesotho.

libsodium (n.d.) Password hashing. Available at: https://doc.libsodium.org/password_hashing (Accessed: 2 October 2026).

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson.

SQLite (n.d.) Appropriate uses for SQLite. Available at: https://www.sqlite.org/whentouse.html (Accessed: 2 October 2026).